Introduction

This worksheet demonstrates the process of performing **linear asymmetric quantisation** on a floating-point matrix we looked at in Efficient AI. Similar to last week, you will do some work implementing your own version of the algorithm, to ensure that you understand the details.

1. Import key packages: torch, and any others that you prefer to work with. In general, when writing code, you will put all your import statements at the top. However, for these worksheets we will import as we go along.

In [ ]:
```
import torch
```

2. Create a random 4×4 Floating-Point Matrix, which we will quantised next.

In [ ]:
```
x = torch.randn(4, 4).float()
print(x)
```

```
tensor([[-0.7433,  0.1364, -0.1520,  0.4991],
        [-0.5750, -0.8882, -0.5028,  1.8397],
        [ 1.2545, -0.0426,  0.5809,  1.6157],
        [-0.6267, -1.6710,  1.5310,  0.7415]])
```

3. Compute Quantisation Parameters (Scale & Zero-point) of this tensor. $ Scale = (x_{\text{max}} - x_{\text{min}}) / (q_{\text{max}} - q_{\text{min}}) $, $ Zero-point =q_{\text{min}} - round(\frac{x_{\text{min}}}{s}) $

In [ ]:
```
# int8 integer range
q_min, q_max = -128, 127

# get data min/max
x_min = x.min()
x_max = x.max()

# compute scale and zero-point
scale = (x_max - x_min) / float(q_max - q_min)
zero_point = q_min - torch.round(x_min / scale)

# clamp zero_point to valid int8 range
zero_point = zero_point .clamp(q_min, q_max)
```

4. Implement the quantisation formula: $q = round(x / scale + zero\_point)$

In [ ]:
```python
def quantise(x, scale, zero_point,q_min,q_max):
  # Implement the quantization formula:
  #   q = round(x / scale + zero_point)
  q = torch.round(x / scale + zero_point)

  # Clamp to valid range:[q_min,q_max]
  q = q.clamp(q_min, q_max)

  return q.to(torch.int8)

# quantisation
x_q = quantise(x, scale, zero_point, q_min, q_max)
print("quantised matrix:\n",x_q)
```

```
quantised matrix:
 tensor([[ -61,    3,  -18,   29],
        [ -49,  -72,  -44,  127],
        [  84,  -10,   35,  110],
        [ -53, -128,  104,   47]], dtype=torch.int8)
```

5. Implement dequantisation formula: $ dequant= (q - zero_point) * scale $

In [ ]:
```python
def dequantise(q, scale, zero_point):
  # Convert q back to float and apply inverse mapping:
  x_dq = (q.to(torch.float32) - zero_point) * scale
  return x_dq

x_dq = dequantise(x_q, scale, zero_point)
print("dequantised matrix:\n", x_dq)
```

```
dequantised matrix:
 tensor([[-0.7435,  0.1377, -0.1514,  0.4956],
        [-0.5782, -0.8949, -0.5094,  1.8449],
        [ 1.2529, -0.0413,  0.5782,  1.6108],
        [-0.6333, -1.6659,  1.5282,  0.7435]])
```